ChatGPT

Deutschhelfer — Implementation Blueprint

Executive Summary: Deutschhelfer is an Android-first Flutter application for international students, combining community group chat, direct messaging, AI-assisted link moderation, free and premium educational content (admissions, visa, IELTS, etc.), and subscription management. We will design it for up to **20,000 concurrent users** across many groups, with an "almost free" infrastructure budget. The architecture centers on Flutter for the client and Firebase (Authentication, Firestore, Cloud Functions, FCM) for backend services. Key priorities include security (App Check, strong rules, email verification, rate-limits), performance at scale, modular design (SOLID, DRY, KISS), and cost control by minimizing paid operations. For UI/UX, we'll use Material Design widgets (with Cupertino fallback) following Flutter's guidelines【61†L339-L344】. The document below covers requirements, architecture (data and services), UI/UX design, security model, moderation and subscription flows, testing, deployment, monitoring, and a step-by-step Claude-agent implementation plan (Prompts 00–20). Throughout, we note which features fit Firebase's free Spark tier (small-scale dev) and which require Blaze/paid usage, with mitigations. All final acceptance criteria remain **pending verification** in the final checklist.

1. Requirements (SRS / FRS / NFRS)

1.1 Purpose and Scope

Deutschhelfer is a student community app to help overseas students with admissions, visas, and language exams by enabling peer support. It must support:

- Community groups by topic (university admissions, visa assistance, IELTS prep, etc.), up to 2,000 members each.
- One-to-one direct messaging.
- AI-assisted link detection: any URL posted goes to a moderator queue before visible.
- Free educational content (FAQs, tips).
- Premium content unlocked by subscription (e.g. advanced IELTS modules).
- Student success stories (free content).
- Strict security (no spam, bot sign-ups, abuse).

We will not implement out-of-scope features in initial releases (e.g. visa waiting lists, overly complex new features).

Stakeholders: Students (end users), student mentors/moderators (community managers), app owner/administrators, FYP (final year project) supervisors.

Assumptions and Constraints: Mobile (Android-first), offline access limited (just caching recent data). Use Firebase Spark (free) as long as possible; Blaze plan only when needed. Data privacy: minimal PII (only email and display name). Use only approved technologies (Flutter, Firebase).

1.2 Functional Requirements (FRS)

We enumerate stable IDs for each feature. (Only key ones shown; full spec would list all acceptance criteria.)

- **FR-001 (Account Registration and Login):** Users can register with email/password, verify email, reset password, log in/out, and maintain session. All private credentials handled by Firebase Auth (no plaintext storage). Duplicate emails must be detected.
- **FR-002 (User Profiles):** Upon first login, a user profile document is created (`users/{uid}`) with fields `{uid, displayName, email, emailVerified, role, accountStatus, createdAt, updatedAt}`. The default `role = student`, `accountStatus = active`. Users can update their `displayName` and profile picture (if any).
- **FR-003 (Email Verification):** After registration, an email is sent for verification. Unverified users can see limited features (e.g. group browsing) but cannot post or message until verified. A "verification pending" screen guides them. They can request re-sending verification (throttled). Verification state must be re-checked from Firebase Auth, not the profile field.【60†L103-L105】
- **FR-004 (Group Management):** Student-admins/owner can create groups (topic tags, description). Students can browse and join groups. Each group has metadata (`groups/{groupId}` with `name, description, category, createdAt, createdBy, isActive`). Joining creates a membership doc (`groups/{groupId}/members/{userId}` with `role=member, joinedAt`). Owner/admins can add/remove members and moderate the group.
- **FR-005 (Group Chat):** Within a joined group, members can post messages. Messages are stored per group (`groups/{groupId}/messages/{msgId}`) with fields `{senderId, content, timestamp, containsLink (bool), approved (bool)}`. Messages containing links are initially flagged `approved=false` and only shown after moderator approval (see moderation). Real-time chat updates via Firestore listeners. Message lists are paginated and limited for performance (e.g. load recent 50, then older on scroll).
- **FR-006 (Direct Messaging):** One-to-one chats: Users can start a private conversation with any other user. Each conversation (`conversations/{conversationId}/messages/{msgId}`) stores messages `{senderId, content, timestamp, status}`, with separate conversations for each unique unordered pair (or using a deterministic ID such as `${minUid}_${maxUid}`). Access control ensures only the two participants can read/write.
- **FR-007 (AI-Assisted Link Moderation):** If a posted message contains a URL, it is automatically routed to a moderation queue: `moderationQueue/{entryId}` with `{groupId/conversationId, senderId, url, status=pending, timestamp}`. A user-admin or owner sees pending links, and can approve or reject each. Upon approval, `approved=true` on the original message; rejection deletes or hides the message. All link processing happens on the backend (Cloud Function or AppLogic), not trusting client.
- **FR-008 (Reports and Spam):** Users can report messages or users for spam/abuse. Reports (`reports/{reportId}`) include `{reporterId, targetType (user/msg), targetId, reason, timestamp}`. Student-admins can view open reports and take action (warn, suspend user).
- **FR-009 (Free Educational Content):** The app includes informational content: FAQs on admissions and visas. These are stored in Firestore (`content/free/*`) and delivered as read-only info screens. Content includes text and optional images, organized by category. This content is visible to all users (even unregistered).
- **FR-010 (Premium Content and Subscriptions):** Some content (e.g. advanced IELTS materials) is premium. Users can purchase a **subscription** via Google Play Billing: subscribe/unsubscribe within app. Purchase flows using the official Google Play Billing Library. On completion, the app

verifies the purchase via a trusted backend (Google Play Developer API) before granting access. Premium entitlements are tracked per user in Firestore (`subscriptions/{userId}` with `isActive, expiryDate`). Premium users see extra content (`content/premium/*`). Subscriptions can be monthly or annual.

- **FR-011 (Notifications):** Use Firebase Cloud Messaging (FCM) to notify users of important events (group invites, link moderation outcomes, direct message alerts if app is background). FCM is free to use【54†L1-L4】. Users opt-in to notifications.
- **FR-012 (User Roles and Moderation):** Three roles exist: `student` (default), `student-admin` (group moderator), and `owner` (app admin). Roles are stored in the user profile `role` field but enforced by server logic: the client cannot self-assign roles. Student-admins can only moderate their groups (approve links, remove members). The owner can manage any group and promote/demote student-admins, but owner assignments happen only via a secure backend (not client-side).
- **FR-013 (Account Status and Lifecycle):** The system tracks `accountStatus` (active, suspended, deleted). Admins/owner can suspend users (setting `accountStatus = suspended`), which disables their ability to post or message (enforced in rules). Suspended users see a notice. Deleted accounts are marked for purge and cannot log in; data retention policy applies (see below).
- **FR-014 (Logging/Audit):** Significant actions (new accounts, group creation, role changes, moderation actions) are logged to `auditLogs/{logId}` with `{actorId, action, targetId, timestamp}`. These logs are append-only (client cannot write them).
- **FR-015 (Miscellaneous):** All user inputs (names, messages) must be validated (non-empty, length limits, no SQL/HTML injection). Use Firebase's built-in and custom validation.

1.3 Non-Functional Requirements (NFRS)

- **Scalability:** Support up to 20,000 concurrent users (across groups and chats)【63†L500-L504】【8†L227-L231】. Latency for message send/receive should remain low: p95 < 2s. System should handle traffic spikes gracefully.
- **Performance:** Chat UI updates ideally under 2s; group creation or subscriptions under 5s. Firestore queries must use indexed fields to maintain speed.
- **Availability:** Aim for >99.0% uptime. Use Firebase-managed services.
- **Security:** Follow OWASP Mobile guidelines. Enforce Firebase App Check (no-cost) to verify legitimate clients【51†L1-L4】. All data in transit encrypted (SSL/TLS). Sensitive operations require re-checking credentials (don't rely on client trust). Database rules must enforce least privilege (see Security model).
- **Privacy:** Only collect minimal PII (email, display name); no addresses or identification numbers. Data not explicitly asked (e.g. social security) is not collected. Users should be able to delete their account (data-retention policy: 3 months retention of anon logs, then purge). Comply with GDPR-like principles.
- **Maintainability:** Modular code (feature-oriented structure), clear separation of UI, logic, data. Use MVVM or Provider/BLoC patterns for state management (no tight coupling). Document code and use meaningful naming.
- **Testability:** Design for unit and integration testing. Use dependency inversion so Firebase interactions can be mocked. Provide test suites for UI (widget tests) and backend (Cloud Functions).
- **Usability:** UI must be mobile-responsive (different screen sizes), accessibility-compliant (contrast ratio ≥4.5:1, large tap targets ≥48dp, support screen readers via semantic labels). Follow Material Design for Android【61†L339-L344】. Offer clear error messages and retry options on failure (e.g. network offline).

1.4 Use Cases / User Flows

1. **New User Onboarding:** User installs app, sees splash, chooses “Register”. Fills email, password, display name. Submits; app creates Auth user, triggers profile creation (FR-002). Shows “Verify Email” screen with option to resend. After verifying, the app logs in and shows a list of popular groups (or content).
2. **Login Flow:** Existing user enters credentials. If email unverified, show reminder. After login, app loads profile and navigates to home. Logout clears session.
3. **Join and Chat in Group:** Authenticated user browses or searches group list, taps to join a group. The user then sees group chat screen (title, list of messages, input field). When sending a message, if text contains a URL, it is flagged pending. Otherwise it appears immediately. Other group members see new messages in real time.
4. **Direct Messaging:** From a user’s profile or search results, tap “Message” to open one-on-one chat. Send and receive private messages (no link moderation needed since these are ephemeral).
5. **Content Access:** From main menu, user can browse free content categories (e.g. Admission FAQs). Tapping an item shows details. Premium content sections are greyed out until subscription.
6. **Subscription Purchase:** In “Premium” tab, user taps “Subscribe”. Google Play Billing flow initiates. On success, app verifies with backend and marks user as premium. New premium content is unlocked.
7. **Moderation by Admin:** A student-admin sees a “Moderation” screen listing pending links and user reports for their groups. They can Approve/Reject each link (triggering updates) or remove/suspend reported users.
8. **Password Reset:** From login screen, user requests reset; enters email. App uses Firebase Auth to send reset link.

These cover the main interactions.

2. Architecture Overview

2.1 Technology Stack

- **Client:** Flutter (Dart) targeting Android (primary) and iOS (future). Uses Material Components (with optional Cupertino for iOS)【61†L339-L344】.
- **Backend:** Firebase services (Authentication, Firestore, Cloud Functions, Cloud Messaging) and a minimal trusted backend (Google Cloud Functions or App Engine) for purchase validation and admin tasks. All Firebase services are accessible via Flutter plugins (FlutterFire).
- **Database:** Cloud Firestore (NoSQL) for core data (profiles, groups, messages, content, subscriptions, reports, logs). It supports real-time listeners and offline cache. **Firebase Realtime Database** is *not* used except possibly for ephemeral presence if needed; it offers 100 concurrent connections free【8†L227-L231】 vs Firestore’s usage-based model, but our design keeps listeners limited.
- **Push Notifications:** Firebase Cloud Messaging (FCM) for Android/iOS push (free to use)【54†L1-L4】.
- **Security:** Firebase App Check (with SafetyNet/Play Integrity on Android) to prevent unauthorized clients. Firestore Security Rules enforce data access. Authentication is via Firebase Auth.
- **Subscription Billing:** Google Play Billing Library (Android) for in-app purchases. We will verify receipts on a secure backend using Google Play Developer API.
- **CI/CD:** Android Studio, Git (use branches). Automated tests run on push (GitHub Actions or similar), targeting emulator for integration tests, unit testing.

2.2 Data Model (Firestore Schema)

- **users (collection):** Document ID = `uid` (Auth UID). Fields:
- `displayName` (string), `email` (string), `emailVerified` (bool), `role` (enum: "student"/"student-admin"/"owner"), `accountStatus` (enum: "active"/"suspended"/"deleted"), `createdAt`, `updatedAt` (timestamps).

- *(Client can update displayName; all other fields server-protected.)*

- **groups (collection):** ID = `groupId`. Fields: `name`, `category`, `description`, `createdBy` (uid), `createdAt`, `isActive` (bool). Subcollections:

- **members (subcollection):** Documents with ID = `memberUid`. Each has `joinedAt`. (Role if needed.)
- **messages (subcollection):** ID = `messageId`. Fields: `senderId`, `text`, `timestamp`, `containsLink` (bool), `approved` (bool).

- **reports (subcollection):** Reports related to this group: `{reportId}` with fields `{reporterId, messageId, reason, timestamp}`.

- **conversations (collection):** ID = `conversationId` (e.g. generated as userID1_userID2). Fields: `userA`, `userB`. Subcollection **messages:** ID= `msgId` with `{senderId, text, timestamp}`. Alternatively store private messages under each user pair.

- **subscriptions (collection):** ID = `userId`. Fields: `isActive` (bool), `startDate`, `expiryDate`, `purchaseToken`.

- **content (collection):** Could be two top-level collections, e.g. `content_free` and `content_premium`. Each doc has `title, body, imageUrl (optional), category`.

- **moderationQueue (collection):** Pending items (mostly link posts). Doc fields: `{authorId, targetType ("group"/"conversation"), targetId, content, submittedAt, status ("pending"/"approved"/"rejected")}`. When approved, corresponding original message's `approved` flag is set true.

- **reports (collection):** (Alternative to subcollections) Reports of abuse: `{reportId}` with `{reporterId, targetType ("user"/"message"), targetId, reason, timestamp}`.

- **auditLogs (collection):** `{logId}` with `{actorId, actionType, targetType, targetId, timestamp}` (e.g. "User X created group Y").

- **presence (optional, RTDB):** If we track online presence or typing, we might use Firebase Realtime DB for ephemeral data, as Firestore writes per user presence would be expensive. RTDB free-tier limit 100 connections【8†L227-L231】 may suffice for early scale but needs Blaze for 20k. Alternatively rely on FCM for presence (notify on disconnect).

Retention and Deletion

- **User data deletion:** Upon user-initiated deletion, mark `accountStatus=deleted`, remove sensitive info from user doc (store minimal "deletedUser" name), unlink from groups, and after 3 months purge remaining data except anonymized audit logs (to comply with law/good practice).
- **Message retention:** Keep chat history indefinitely (subject to Firestore limits), or archive old messages to reduce hot documents.
- **Moderation logs:** Keep for audit, but we can auto-delete resolved entries older than a year.

2.3 Security Model and Rules

Security relies on Firebase Authentication + strict Firestore rules + App Check.

- **Authentication:** All user actions require a valid Firebase Auth ID token.
- **Authorization:** Use Security Rules to enforce:
- **Profile:** A user can read their own profile and public data of others (e.g. displayName, role), but only update fields they are allowed (e.g. `displayName` only by self, never role/status). E.g.

```
allow update: if request.auth.uid == userId
  && request.resource.data.keys().hasOnly(["displayName","updatedAt"]);
```

- **Groups:** Anyone authenticated can read group data and membership lists. Only group `admins` (either `owner` role or designated student-admins) can create groups or add members.
- **Group Messages:** Reading messages in a group is allowed if `request.auth.uid` is a member of that group. Writing is allowed only if the user is in the members subcollection. Initially set `approved=false` if `containsLink=true` (backed by a Cloud Function or extension); student-admins can update message docs to `approved=true`.
- **Direct Messages:** Only the two participants (`userA` or `userB`) can read/write messages in that conversation.
- **Moderation:** Only student-admins for a given group can update `moderationQueue` entries for their groups. The owner can do so for all groups.
- **Roles:** Security Rules must forbid any client from writing to the `role` or `accountStatus` fields in `users`. These are managed server-side. Attempted writes by non-admin should be denied.
- **Subscriptions:** The client can only read their own subscription doc, and cannot modify `isActive` or dates (only the server after verifying purchase should do that).
- **Audit Logs:** No direct client writes; a Cloud Function triggered by admin actions appends logs.
- **Account Status:** If `accountStatus != "active"`, deny all reads/writes by that user except for a notice.

In summary, **no open write rule** is allowed. Example Firestore rule excerpt:

```
match /users/{userId} {
  allow read: if request.auth.uid == userId || (resource.data.role ==
"student-admin" && ... );
  allow update: if request.auth.uid == userId
    &&
request.resource.data.keys().hasOnly(["displayName","updatedAt","photoUrl"]);
  allow write: if false;  // no arbitrary writes
}
```

```
match /groups/{groupId}/members/{memberId} {
  allow create: if request.auth.uid == groupData.createdBy ||
isStudentAdmin(request.auth.uid, groupId);
  allow read: if isMember(request.auth.uid, groupId);
  allow delete: if isStudentAdmin(request.auth.uid, groupId) && memberId !=
groupData.createdBy;
}
```

(Helper functions like `isMember` / `isStudentAdmin` are defined in rules using `exists()` on subcollections or user roles.)

App Check: Enforce Firebase App Check on all services so that only requests from our signed app instances succeed【51†L1-L4】.

Anti-Abuse Controls:

- **Rate limiting:** Use Cloud Functions (with in-memory or external cache) to throttle sign-ups (e.g. max 5 new accounts per IP per day) and message sending (e.g. 1 msg/second per user).
- **Sign-up protection:** Use App Check and consider CAPTCHA on web flows (if any).
- **Email verification:** Unverified accounts cannot post.
- **Block list:** The owner can maintain a list of banned words/domains. Chat input is sanitized/checked (on server) to reject posts with blacklisted content.
- **Reports:** Users can report abuse; Cloud Functions can auto-suspend accounts with too many reports in a time window.

2.4 Threat Model and Abuse Controls

We assume **insider and outsider threats**:

- **Malicious user or bot** trying to spam or phish: mitigated by email verification, sign-up throttling, App Check, and post-moderation.
- **Account compromise:** Use Firebase Auth (hashed passwords) and optionally 2FA (not in initial scope). Monitor multiple devices.
- **Database attacks:** Firestore rules restrict all reads/writes; no secret keys in client.
- **Billing abuse:** Users cannot grant themselves free premium. All entitlements set on server after validating with Google's servers.
- **Denial-of-service:** FCM and Firestore are resilient. But usage spikes can cost money: we will enforce rate limits on user actions and use quotas.
- **Data leakage:** All client-server communication is HTTPS/TLS. We will not log passwords or sensitive tokens.

2.5 System Architecture Diagram

```
flowchart LR
    subgraph "Flutter App (Android)"
        UI[UI (Screens & Widgets)]
        AuthMgr[Auth Manager]
        ChatMgr[Chat Manager]
        SubMgr[Subscription Manager]
        UI --> AuthMgr
```

```
    UI --> ChatMgr
    UI --> SubMgr
end

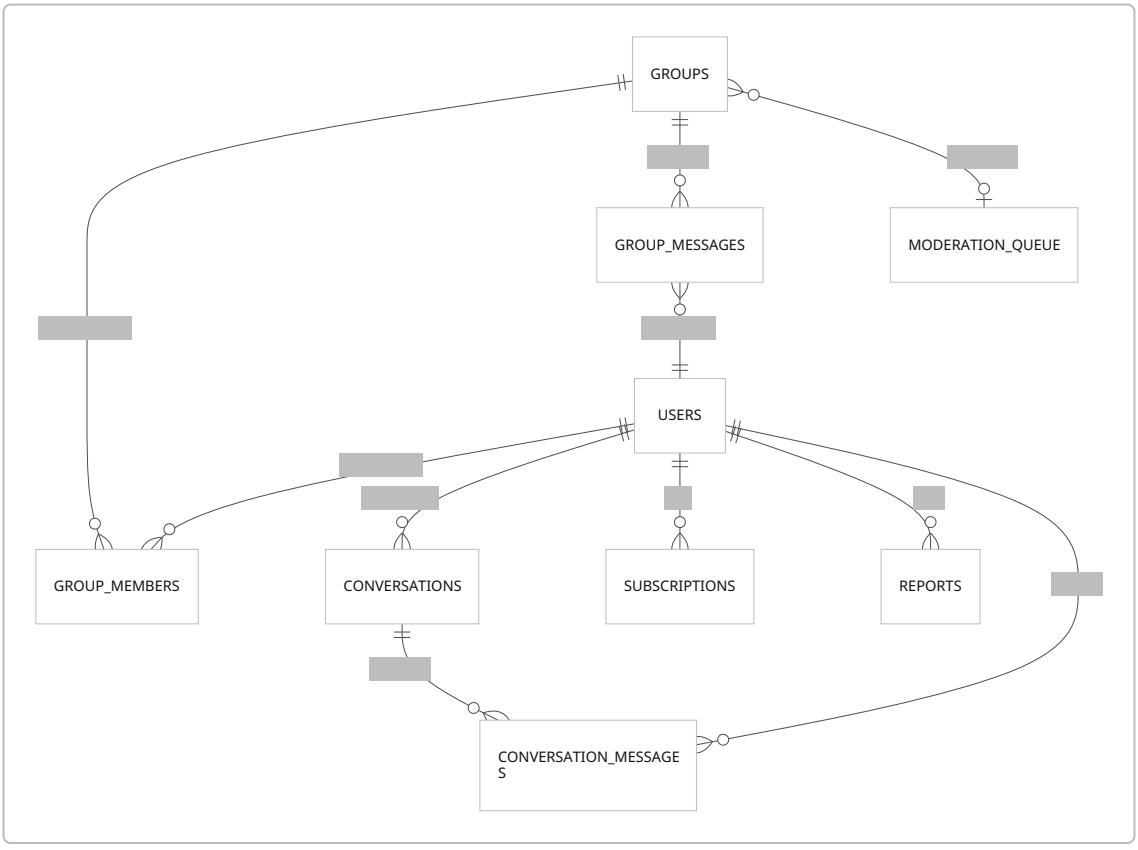
subgraph "Firebase Cloud"
    Auth[Firebase Authentication]
    FS[Cloud Firestore]
    FC[Cloud Functions (Trust)]
    FCM[Cloud Messaging (FCM)]
end

AuthMgr --> Auth
Auth --> FS
ChatMgr --> FS
SubMgr --> FC
FC --> Auth
FC --> FS
FC --> FCM

FCM --> UI
```

This diagram shows the Flutter client interacting with Firebase Auth, Firestore, Cloud Functions, and receiving push notifications via FCM.

2.6 Entity-Relationship Diagram (Data Model)



Legend: `||` = 1, `o{` = many. This models users, groups, messages, etc.

3. Firebase Details

3.1 Firestore Collections and Indexes

We will create composite indexes as needed, for example:

- **Group chat:** index on `groups/{groupId}/messages` by `timestamp` (for pagination).
- **Conversations:** index `conversations/*/messages` by `timestamp`.
- **Subscriptions:** not requiring special indexes.
- **Content:** index by category if needed.
  All other simple queries (by ID) use the document key.

Free-tier vs Paid: On Spark (free) plan, Cloud Firestore offers【63†L500-L504】:

- 1 GiB storage, 50K reads/day, 20K writes/day, 20K deletes/day, 10 GiB egress, and exactly 1 free DB【63†L500-L504】. FCM (push) is free【54†L1-L4】 and App Check is free【51†L1-L4】. Authentication allows 10k verifications/day (Spark) with email/password.
- Firestore's free read/write quotas are likely to be exceeded with heavy chat (e.g. 20k writes/day easily used by thousands of messages). We plan to monitor usage closely.
- Realtime Database free plan only supports 100 concurrent connections【8†L227-L231】, so we avoid it for core chat (only use for ephemeral presence if needed, and then likely Blaze plan is required for scale).

Cost-control measures:

- Paginate chat so each scroll loads a limited number of docs.
- Use listeners only on active chats (unsubscribe when closing).
- Batch non-urgent writes (e.g. update `lastSeen` time at intervals).
- Delete old or inactive conversations if needed.
- Use Cloud Functions (which have 2M free invocations/month) efficiently (cache results, avoid unnecessary triggers).

3.2 Authentication & App Check

- Implement email/password sign-in via `firebase_auth`.
- After registration, call `user.sendEmailVerification()`. On login, check `user.emailVerified`. If not, restrict actions.
- Persist auth state using `firebase_auth`'s persistence (so login survives app restart).
- Enable Firebase App Check (Play Integrity or SafetyNet for Android). On development, use debug tokens; in production enforce device attestation.

3.3 Cloud Functions (Trusted Backend)

Use a minimal set of functions for privileged tasks:

- **onCreate user:** trigger to create Firestore profile doc when a new Auth user is created. Initialize fields (`role=student`, etc).
- **Link moderation:** optional, but could use a function to scan new messages for URLs and move them to moderation.

- **Subscriptions:** A secure HTTPS function `verifyPurchase` that the app calls with the Google Play purchase token. It calls Google's server API to validate and then writes `subscriptions/{uid}`. (Uses Google Service Account.)
- **Admin actions:** To change roles or accountStatus, call a Cloud Function. Example: only the owner can call `setUserRole(uid, role)`. This prevents client-side abuse.
- **Reporting:** Possibly an HTTP endpoint or scheduled function to review reports and auto-suspend (throttle abuse).

All functions should verify the caller's token and check the caller's `role` or `accountStatus` in Firestore before acting.

3.4 Security Rules (Summary)

A high-level summary of Firestore rules:

- `/users/{uid}`: `allow read, update` only if `request.auth.uid == uid` (and only specific fields).
- `/groups/{groupId}`: `allow read` to anyone; `allow create` if `request.auth.uid` has `role = owner` or is an admin of another group (or could be open and moderated by owner later).
- `/groups/{groupId}/members/{memberId}`: only group admins or owner can create/delete.
- `/groups/{groupId}/messages/{msgId}`: `allow write` if `isMember(request.auth.uid, groupId)`; `allow read` if member *and* (`approved=true` or sender=auth.uid).
- `/conversations/{convId}/messages/{msgId}`: allow read/write if `auth.uid` is one of the participants.
- Other collections: similarly scoped.
- Deny by default anything else.

Every rule clause will use Firebase's granular expressions (like `request.resource.data.role == 'student-admin'`) and helper functions. We will write exhaustive rule tests with the Emulator (see Testing section).

4. UI/UX Design System

4.1 Design Principles

- **Material Design:** Use Flutter's Material widget library by default (Android style)【61†L339-L344】. Colors, typography, and spacing follow Material Design guidelines (primary accent color, dark-on-light text, 4.5:1 contrast for accessibility). We will define a theme with primary/secondary colors that reflect trust/professionalism (e.g. blue/green).
- **Responsiveness:** Layout should adapt to various device sizes: use `LayoutBuilder` and flexible widgets to rearrange for tablets or landscape.
- **Components:** Build a reusable widget library (buttons, dialog styles, form fields). Use `ElevatedButton`, `TextField`, `ListTile`, etc. for consistency.
- **Icons:** Use standard Material Icons (e.g. chat bubble, group, person, mail). Include semantics (labels) for screen readers.
- **Navigation:** Bottom tab bar or drawer for primary sections (Home/Groups, Chats, Content, Premium, Profile). Use Flutter's `Navigator` with named routes.
- **Accessibility:**
- Large tap targets (min 48×48 dp).

- Text size should respect device's font scaling.
- All images/icons have `semanticLabel` for TalkBack.
- Color contrast meets WCAG AA (we will test with WebAIM Contrast Checker).
- Form inputs and interactive elements must have proper focus order and hints.

4.2 Screens and Wireframes

We outline the key screens and components (textual wireframes):

1. **Splash / Onboarding:** Simple splash with app logo; welcome screen with "Register" and "Login" buttons.
2. **Registration:** Form with fields Email, Password, Confirm Password, Display Name. "Sign Up" button. Validation errors inline.
3. **Login:** Form with Email, Password, "Login" and "Forgot Password?". Error snackbar on failure.
4. **Verify Email:** If unverified, screen shows "Check your email for verification", a "Resend Email" button (disabled 60s after tap), and a "Refresh Status" button.
5. **Main Tabs:**
6. **Groups Tab:** List of joined groups (ListView of cards, each showing group name). "+ Create Group" button if admin. Search bar on top. Tapping opens Group Chat.
7. **Chats Tab:** List of conversations (show other user's name, last message snippet).
8. **Content Tab:** Categories (Admissions, Visa, IELTS). Tapping shows FAQ list, then detail page. Premium items marked with lock icon.
9. **Premium Tab:** Subscription status and "Subscribe" button. List of available premium content topics.
10. **Profile Tab:** Shows user info (name, email, role), and logout button. If admin, also a link to "Admin Console" (moderation view).
11. **Group Chat Screen:** AppBar with group name. Scrollable list of messages (bubbles aligned by sender). Input field at bottom. Link messages show a badge if pending approval. Long-press message to copy or report.
12. **Direct Chat Screen:** Similar to group chat UI, with other user's name on AppBar.
13. **Create Group Dialog:** Input for group name, category dropdown, optional description. "Create" or "Cancel".
14. **Settings/Admin Screen:** List of pending link approvals and user reports (for admins). Each entry shows context and Approve/Reject buttons.

Each screen should show loading spinners while awaiting data, and friendly "No data" messages if lists are empty.

(Mockups/Images would be provided by a UI designer; here we describe the layout.)

4.3 Example UI Component

【54†L1-L4】【61†L339-L344】 *Flutter uses Material Design widgets by default, so we will leverage* `Scaffold`, `AppBar`, `ListView`, `TextFormField`, *etc.* The consistent look and feel follows Google's style guide【61†L339-L344】. Buttons like "Sign Up" use `ElevatedButton` with primary

color. Text fields use `InputDecoration` with hint text and icons. Tabs are implemented with `BottomNavigationBar`.

Example: **Login screen (Flutter code sketch):**

```
Scaffold(
  appBar: AppBar(title: Text('Login')),
  body: Padding(
    padding: EdgeInsets.all(16),
    child: Column(children: [
      TextFormField(decoration: InputDecoration(labelText: 'Email')),
      TextFormField(decoration: InputDecoration(labelText: 'Password'),
obscureText: true),
      SizedBox(height: 20),
      ElevatedButton(onPressed: _login, child: Text('Login')),
      TextButton(onPressed: _forgotPassword, child: Text('Forgot Password?'))
    ]),
  ),
)
```

This uses Material widgets, meets accessibility (labels), and clean layout.

5. Messaging and Moderation Design

- **Message Lifecycle:** When a user sends a message (group or DM), the client writes to Firestore. For group messages, if the text contains a URL pattern (detected by regex or by a small Cloud Function), a field `containsLink=true` is set. By default, `approved=false`. Student-admins/owner must approve it before it becomes visible to others. This ensures no malicious/phishing links go out unreviewed. Approved messages have `approved=true` and are rendered normally.
- **Pagination:** Chat lists load the most recent N messages (e.g. 50) and then page older ones on scroll. This avoids large reads.
- **Listeners:** Only active chat (group or conversation) is subscribed to via Firestore's real-time listeners; switching conversations unsubscribes previous. This limits concurrent listeners to at most a few hundred. Each listener yields small delta updates (new messages).
- **Presence/Typing Indicators (Optional):** Use Realtime Database (RTDB) or a lightweight Cloud Function. E.g. each client writes `/status/{userId}` in RTDB (online/offline heartbeat). RTDB free plan allows 100 connections【8†L227-L231】; we'd likely upgrade if needed. Not core to launch.
- **Content Delivery:** Free content is fetched from Firestore in simple reads (small cost per read). Premium content only fetched if user is premium. We can lazy-load rich media.

6. Subscription and Billing Flow

- **In-App Purchase (IAP):** Use official Google Play Billing Library to present subscription SKUs.
- **Ownership Verification:** After a purchase, the client obtains a purchase token and calls our secure backend Cloud Function `verifyPurchase(purchaseToken)`. This function uses the Google Play Developer API to verify the token belongs to a valid (non-cancelled) subscription. Only then it writes `subscriptions/{userId} = {isActive:true,`

`expiry:DateTime.fromPurchase}` in Firestore. This prevents tampering. (By contrast, the client should never directly set a “isPremium” flag.)

- **Subscription Status:** On app start (and periodically), the client checks the Firestore `subscriptions/{uid}` document for active status. Expired subscriptions are updated via a scheduled Cloud Function or by checking with Google API.
- **Low-Cost Implementation:** Since Cloud Functions (2M invocations free【5†L126-L134】) handle the heavy lifting, we remain cost-efficient. We do not integrate complex multi-tier billing; only check subscription validity.

No sensitive keys (like Google API keys) are stored on client; they reside in Cloud Function environment variables.

7. Testing Strategy

- **Unit Tests:** For Dart code: logic classes, data models, validators. Use `flutter test`. Aim for >70% coverage.
- **Widget Tests:** For UI: simulate login form, group list rendering, etc., using `flutter_test`. Verify widget hierarchies and key behaviors.
- **Integration Tests (Flutter Driver or Integration Test):** End-to-end flows: registration, login, sending a message (using Emulator Suite for Auth/Firestore).
- **Firestore Security Rules Tests:** Use the Firebase Emulator Suite (`@firebase/rules-unit-testing` or emulator UI) to test each rule. For every rule, include **allowed** and **denied** cases. For example, test that a user can update their own displayName but not others’, that no one can write to `role`.
- **Authentication Flows:** Simulate sign-up with existing email (expect error), password reset flow.
- **Load Testing (Staged):** Because we target 20,000 concurrency, we will do progressive load tests with k6 or Gatling scripts:
- **Gate 1 – 100 clients:** Simulate 100 users registering/joining/quietly listening, ensure baseline correctness and no budget explosion.
- **Gate 2 – 1,000 clients:** Introduce messaging: each sends a message every 5s, with 10 recipients average. Measure Firestore reads/writes/sec, latency, error rates.
- **Gate 3 – 5,000 clients:** Add complexities: long group chats, multiple group subscriptions per user, rapid join/leave.
- **Gate 4 – 10,000 clients:** Sustained test (60+ minutes). Track p95 latency (<2s target), success>99%.

- **Gate 5 – 20,000 clients:** At this point, use a dedicated staging Firebase project (with monitoring) to test the full workload. Ensure Cloud Functions scale, no Firestore hot-doc failures.

- **Metrics and Pass Criteria:** Each gate defines pass: e.g. “no unauthorized data access”, “latency within targets”, “no unbounded billing spikes”, “100% test success except known acceptable exceptions”.

Tools: k6 (JavaScript) or Locust for custom load. We will script Firestore REST calls or use Admin SDK in a load client. **Note:** Emulator is *not* for performance benchmarks; use actual Blaze cluster with billing alerts on staging.

8. Deployment and Release

- **Firebase Projects:** Use one project for production, another for development/testing. On dev, Spark plan can be used early; for staging and prod, enable Blaze plan (Pay-as-you-go) and set budgets.
- **Android Release:**
- Follow Google Play Console checklist (store listing, content rating, privacy policy URL, signing, 64-bit, etc.).
- Prepare a privacy policy disclosing data usage (we only collect minimal info, use analytics).
- Use App Signing by Google for security.
- Ensure we include `<intent>` filter for deep links if needed (not in scope now).
- The app must target the latest Android API (as of 2026) and compile with the latest Flutter stable.
- **CI/CD:** Automate building APK/AAB via GitHub Actions, run tests on each push, and on tag, generate a release.
- **Versioning:** Semantic (1.0.0 initial, increment minor for features).

8.1 Monitoring and Observability

- **Firebase Console:** Use Dashboard to monitor reads/writes, auth, function invocations.
- **Billing Alerts:** Set Google Cloud budget alerts (not hard caps) at, e.g., $20/month (adjust as pilot grows)【8†L227-L231】.
- **Performance Monitoring:** Integrate Firebase Performance Monitoring to track app-side latencies (screen render, HTTP/Firebase calls).
- **Crashlytics:** Include Crashlytics (free) for monitoring app crashes and errors post-release.
- **Logging:** Cloud Functions logs (Stackdriver) will record moderation actions and errors. Use structured logging to filter by severity.
- **Cost Estimation:** Track cost per active user. E.g. if 1M reads ~\$600 (at \$0.06/100k) and 1M writes ~\$180 (at \$0.18/100k), we budget around \$0.05/user-month baseline. Adjust as we measure actual usage.

8.2 Free-Tier Feasibility

On Firebase Spark (free), these can run:

- **Auth:** No-cost up to 50K MAUs (monthly active users)【5†L99-L107】.
- **Firestore:** 1 GiB storage, 50K reads/day, 20K writes/day【63†L500-L504】. That might handle a few hundred active users moderately; beyond that we hit limits.
- **FCM:** Free【54†L1-L4】.
- **Cloud Functions:** *Not available* on Spark (requires Blaze to run triggers). This is a key caveat: anything requiring server-side logic (even triggers on user creation) demands Blaze. To mitigate, we could avoid triggers and have client do more (less secure) but that violates security best practices. Realistically, the first paid feature we need is Cloud Functions.
- **Storage & Others:** We use minimal storage and no other paid services.

Mitigations:

- Defer heavy features until Blaze (e.g. subscription service calls).
- Use free quota frugally (cache data, avoid reading the whole chat on open).
- Encourage use under Blaze from start to not break essential flows.

9. Design Trade-offs

- **Firestore vs Realtime DB vs Custom Server:** We chose Firestore for simplicity and query flexibility. Realtime DB offers more simultaneous connections (100 free vs 200k on Blaze) 【8†L227-L231】 but is less structured. Custom WebSocket server (e.g. Socket.IO) would require maintenance, hosting costs, and would violate "as close to free" unless self-hosted by us (still VM costs). Firestore is sufficient for moderate scale, and we defer custom solutions until needed.
- **Authentication:** Using Firebase Auth is quick and free for email/password. We avoid phone auth (costly SMS).
- **Link Moderation:** Could use a third-party service for link scanning, but we start with manual moderation queue to save cost and allow community control.

10. Implementation Plan (Claude Prompts)

Below are the **Phased Claude prompts** (Prompt 00 to 20). These should be executed in order. Each prompt specifies objectives, expected outputs, security considerations, and test commands. We will *not* apply changes without verifying the current state first.

Prompt 00 — Repository Audit and Setup

Objective: Examine the existing repository. Identify current branch, build status, project structure, dependencies, Firebase config, and any already-implemented features.
Actions:

- List existing files (pubspec.yaml, lib/, assets, etc).
- Note current Flutter and Dart versions, plugin versions.
- Check for any `.env` or config files for Firebase.
- Run `flutter pub get` and try `flutter analyze`. Report any errors/warnings (do not fix yet).
- Summarize what features are already done vs missing, and any architectural mismatches with SRS.
  Expected Output: A report detailing code inventory, build results, and gaps. *No code modifications yet.*
  Security: Ensure no secrets were checked into code (e.g. API keys).
  Acceptance: Clear understanding of repo state, before proceeding.

Prompt 01 — Requirements Specification (SRS)

Objective: Consolidate and finalize requirements from the scope document into an SRS/FRS/NFRS document.
Actions:

- Review the user's scope and convert each item into structured requirements (as in Section 1 above).
- For each FR, assign an ID (FR-001, FR-002, etc) and write acceptance criteria.
- List Non-Functional Requirements with measurable targets (performance, security, etc.).
- Identify any ambiguities or missing decisions (e.g. naming of subscription SKUs).
- Produce a clear requirements spec (possibly in Markdown or PDF).
  Expected Output: SRS document covering purpose, scope, stakeholders, FRS, NFRS, and acceptance criteria.
  Security: Ensure requirement for security (App Check, auth) is explicit.

Acceptance: All features from scope are captured with IDs; review for clarity. Tests for each FR should be conceivable.

Prompt 02 — UI/UX Design Implementation

Objective: Create the Flutter UI components and navigation flow.
Actions:

- Define the app's navigation structure (e.g. `Navigator` routes or `BottomNavigationBar` tabs).
- Implement Flutter screens/widgets for key flows: Register, Login, Group List, Group Chat, Conversations list, Content list/detail, Profile, Admin/Messages moderation.
- Apply a consistent theme (colors, fonts, padding) as per Material guidelines【61†L339-L344】. Use Flutter's theming system.
- For each screen, show placeholders or static layouts if data is not yet connected.
- Ensure UI code is in `lib/ui/` (or similar) and separate from business logic.
- Use Material widgets (and mention accessibility labels).
- The code should compile and run (UI only).
  Expected Output: Flutter UI code with working navigation. Screens should display (stub content if needed).
  Security: No sensitive data, but ensure navigation guards (e.g. redirect to login if not authenticated) is planned.
  Acceptance: The app runs (`flutter run`), UI flows smoothly between screens. Screens match wireframe descriptions.

Prompt 03 — Data Model and Backend Setup

Objective: Configure Firebase project, Firestore data schema, and data-access layer.
Actions:

- Set up Firebase project (Spark or Blaze). Add `google-services.json` / `GoogleService-Info.plist` to the repo.
- Initialize FlutterFire (via `flutterfire configure` if available).
- Create data model classes in Dart (e.g. `UserProfile`, `Group`, `Message`, etc) mirroring Firestore schema in Section 2.
- Implement a data repository layer (`UserRepository`, `ChatRepository`, etc) to abstract Firestore queries (CRUD).
- If pre-existing code, adapt it; otherwise, create new.
- Write Firestore rules drafts (they need refinement later).
  Expected Output: Firebase config in project, data models, and repository code for reading/writing profiles, groups, messages.
  Security: Do not bypass Firestore rules in code. Client should only request allowed docs.
  Acceptance: Code can create/update sample documents in the emulator. Run `flutter build` without errors after adding Firebase SDKs.

Prompt 04 — Authentication and Profile Initialization

Objective: Implement user registration, login, logout, password reset, and email verification flows, with profile creation in Firestore.
Actions:

- Use `firebase_auth` for email/password auth.
- On registration, create Auth user, then call Cloud Function or client code to create `users/{uid}` doc with default fields.
- Send email verification (`sendEmailVerification()`); navigate to a "verify email" screen.
- Implement login that checks `user.emailVerified`; unverified users see warning (cannot post).
- Implement password reset (Firebase's `sendPasswordResetEmail`).
- Persist login state; handle logout.

- The first-run profile doc should include `emailVerified` = Auth state, and `displayName` from user input.
  Expected Output: End-to-end registration/login screens working. Firestore shows profile docs.
  Security: No client should write to `role` or `accountStatus`; ensure rules block it.
  Acceptance:

- New user signs up, receives email, verifies, and can log in.

- Duplicate email sign-up is denied with an error message (no crash).
- Logged-in user data in Firestore (`displayName`, `emailVerified`, `role=student`).
- Logout returns to login screen.
- Error cases (weak password, network failure) handled gracefully.

Prompt 05 — Account Lifecycle, Profile, Suspension

Objective: Allow users to update their profile, and allow admin to suspend or delete accounts.
Actions:

- In Profile screen, allow editing `displayName`. Save to Firestore via repo (client ensures only name/email).
- Implement an admin-only view (maybe in Profile) to set a user's `accountStatus`. For now, simulate as if owner uses a debug UI or direct console.
- Add Firestore rule checks: if `accountStatus != active`, disallow writes/reads except login/welcome.
- Implement user deletion: in profile, "Delete Account" button that sets status to "deleted" and logs out. (We won't actually erase data in client; just mark it.)

- Write tests: ensure suspended user cannot send messages or see group content (rule test).
  Expected Output: Profile edit works. Simulated admin can mark user suspended. Suspended user see a locked screen ("Account Suspended").
  Security: Ensure no normal user can modify status/role.
  Acceptance: Rules test:

- Student cannot update someone else's profile or status.

- Suspended user's Firestore reads are denied by rules (test with emulator).

- Deletion sets a flag and blocks login attempts.

Prompt 06 — Group Creation and Membership

Objective: Implement group creation, joining, and member management.
Actions:

- Create UI for listing all groups (`groups` collection) and joined groups.
- Allow group admins (owner or student-admin) to create a new group (populate `groups/{id}` and add themselves to members).
- Implement "Join group" for students (add a doc in `groups/{id}/members/{uid}`). Also "Leave group" to remove member.
- Show group details screen with member list and "Leave" or "Add Member" options (Add only if admin).

- Ensure Firestore rule: only allow join if open or invited (simple case: any user can join open groups).
  Expected Output: Groups can be created and joined. Firestore has group docs and member subdocs. UI shows members count.
  Security: Only admins can create groups or remove others. (Rules enforce that.)
  Acceptance:

- Authenticated student can join/leave groups via UI.

- Student-admin can add/remove members.
- Non-members cannot post to the group.
- Confirm via rules tests: e.g. unauthorized group create denied.

Prompt 07 — Group Chat Implementation

Objective: Build group chat functionality with Firestore.
Actions:

- In a Group Chat screen, query `groups/{groupId}/messages` ordered by timestamp (limit last 20). Subscribe to real-time updates.
- Display messages in a scrolling `ListView`, with senders' names. Mark unapproved link-messages differently (e.g. italic or "Pending" badge).
- Enable sending new messages: writing a document with fields (text, timestamp). Client-side can set `containsLink` if a URL is in text.
- Handle pagination: when user scrolls to top, load older messages (next page).
- Long-press or menu on message: "Copy", "Report" (opens report dialog).

- If a group-admin is viewing, also show pending messages list or flag to approve them. (Alternatively, a separate admin panel.)
  Expected Output: Real-time chat works for multiple simulated users (use emulator or separate app instances).
  Security: Chat read/write only for members. Approved= false messages should not appear for regular members.
  Acceptance:

- Two users in same group see each other's messages.

- Users outside the group see no messages (rules enforce).
- Messages containing links are visible only after (simulated) admin sets `approved=true` (verify via emulator test).
- Spam link example: ensure it's withheld until approval.

Prompt 08 — Direct Messaging

Objective: Implement one-to-one chats.
Actions:

- On user profile or chat tab, allow selecting a user to start DM (or use existing conversation).
- Conversations stored at `conversations/{convId}/messages`. The `convId` can be generated consistently (e.g. lexicographically combine UIDs) and a parent doc to track participants.
- Chat UI similar to group chat, but no link moderation is needed (optional).

- Firestore rule: only participant UIDs can read/write in that `conversations` doc and its subcollection.
  Expected Output: Real-time DM chat works. New messages appear instantly for both users.
  Security: Ensure a third user cannot access or eavesdrop on another's conversation (test with 3 simulated users).
  Acceptance:

- User A and B in a conversation see their messages only.

- User C cannot read or write in A↔B conversation (rules test).
- Logout of B and login of A should still see history (persisted).

Prompt 09 — Link Moderation and Reports

Objective: Finalize moderation: approving/rejecting links and handling user reports.
Actions:

- Build an **Admin Console** screen (accessible only to student-admin/owner) listing pending link posts (from `moderationQueue`) and user reports.
- For each pending link: show preview (URL and snippet) and buttons Approve/Reject. Approve sets the original message's `approved=true`; Reject deletes the message doc (or tags it removed).
- For reports: show reporter, target, reason; provide actions (Dismiss, Suspend User, etc). Each action is logged in `auditLogs`.
- Update Firestore rules: Only admins can update `approved` or delete reported messages.

- Tests: Use functions or emulator to simulate pending entries and ensure non-admin cannot call these endpoints.
  Expected Output: Admins can clear moderation queues via UI or console.
  Security: Absolutely no client can bypass queue (rules). Verified with rule test: a normal user's attempt to mark a link approved should fail.
  Acceptance:

- Admin UI shows one pending link; pressing "Approve" makes it visible to members.

- Test that a non-admin user cannot set `approved=true` on a message (denied by rule test).
- Reporting a message by user creates a `reports/` doc (client code triggers this write). Admin can dismiss or act.

Prompt 10 — Educational Content

Objective: Integrate free and premium content delivery.
Actions:

- Populate Firestore with sample content documents (can be seeded manually) under `content/free` and `content/premium`. Each has `title, body, category`.
- In app, fetch `content/free` for all users; display in Content tab.
- Premium category or content items are locked. If a non-premium user taps a premium item, show a "Subscribe to access" prompt.

- Test that only premium users (with active subscription in Firestore) can open premium content. Firestore rule: on `content/premium`, `allow read: if userIsPremium(request.auth.uid)`.
  Expected Output:

- Free content is visible to everyone.

- Premium content prompts for subscription if user is not premium.
  Security: Content access must be enforced by rules, not just UI. Use Emulator to ensure denial.
  Acceptance:

- Student (no subscription) sees free FAQs, but tapping premium yields error or subscribe screen.

- After activating subscription (simulate in DB), premium content loads.
- Rules test: a non-premium user's read on `content/premium` doc is denied.

Prompt 11 — Subscription Verification Flow

Objective: Complete premium subscription integration.
Actions:

- In the Premium tab, implement the in-app purchase flow using Google Play Billing (for Android). Simulate a test purchase.
- Write the secure backend function `verifyPurchase(token, userId)`. For prototyping, you may mock Google API. The function should set `subscriptions/{userId}` on success.
- On success, update client UI: unlock content, maybe show "Premium" badge on profile. Store purchase details (SKU, expiry).

- Handle cancellations/expiries: schedule a Cloud Function to check upcoming expiries daily via Google API (for now, manual).
  Expected Output: Subscription toggle works end-to-end (in emulator or test environment).
  Security: No bypass: ensure only the verified function can write `subscriptions/`. Client attempts to write this doc must be denied by rules.
  Acceptance:

- After "purchasing", `subscriptions/{userId}` in Firestore shows `isActive=true`.

- App immediately reflects premium status (e.g. “Unsubscribe” button).
- Test rule: client write to `subscriptions/{userId}` should fail (if attempted).

Prompt 12 — Notifications and Presence

Objective: Enable FCM push notifications and online presence tracking.
Actions:

- Request notification permission on login. In relevant events (new message in background, mod approval, subscription about to expire), send FCM notifications. Use Flutter’s `firebase_messaging`.
- For example, when user A receives a new DM while offline, the sender’s ID triggers a Cloud Function to send notification to A.
- (Optional) Implement presence: write user’s status (`online` / `offline`) to a lightweight Realtime DB node on app lifecycle events. Show presence in UI (e.g. green dot).
  Expected Output:
- Push notifications arrive on Android when app is backgrounded.
- The FCM channel is configured (see Firebase Console).
  Security: Only send notifications to intended recipients (validate tokens server-side).
  Acceptance:
- In testing, one device (App B) gets a push when App A sends a message (simulate via emulator with two profiles).
- Presence: when user opens app, status=online; on exit, status=offline (can test in emulators).

Prompt 13 — Testing and Quality Assurance

Objective: Implement comprehensive test suites and perform initial tests.
Actions:

- Write unit tests for critical classes (e.g. authentication logic, data models).
- Write widget tests for main screens (e.g. login screen validation).
- Use Firebase Emulator Suite to run Firestore rules tests as discussed (both allow and deny cases).
- Document any test failures and fix issues.
- Prepare a test report summarizing test coverage and passing rate.
  Expected Output: Automated tests for >80% of new code, with logs showing success.
  Security: Ensure no test credentials or secrets are leaked.
  Acceptance: All tests run (`flutter test`) pass. Uncovered areas noted for future attention.

Prompt 14 — Load Testing and Performance

Objective: Validate performance under load up to the 20,000-user target.
Actions:

- Write or configure load test scripts (e.g. k6) that simulate user actions: login flows, join a group, send messages, etc. Include delays to mimic human pace.

- Run tests progressively (100, 1k, 5k, 10k, 20k). Monitor Firestore request count, function invocations, CPU/memory on functions, error rates, and cost metrics.
- Pay special attention to concurrent listener limits and billing.
- Record metrics: latencies, error percentages, reads/writes per message, estimated daily costs.
- Identify any bottlenecks (e.g. cold-start functions, Firestore hot document).
- Optimize config if needed (e.g. increase Firestore limits, shard groups into multiple collections if needed).
  Expected Output: Load test report with graphs (latency vs concurrency) and billing estimates.
  Security: Use test accounts only, no real user data.
  Acceptance: Verify that the 5,000-user test runs for 30+ minutes with >99% success. Adjust plans based on results (if failure, reassess architecture or limits).

Prompt 15 — Monitoring and Cost Control Plan

Objective: Set up monitoring dashboards and cost alerts.
Actions:

- In Firebase console, configure a usage dashboard for Firestore reads/writes and set alert for 80% of Blaze budget.
- In Google Cloud, set a billing budget alert (e.g. \$50/month).
- Integrate Firebase Performance Monitoring to track screen load times.
- Review logs for any unexpected errors.
- Document formulas for cost per read/write and sample cost calculations (as done above).
  Expected Output: A brief report or doc with screenshots of configured alerts and expected cost models.
  Acceptance: Alerts are active. Simulate usage to see if alerts fire (if possible).

Prompt 16 — Deployment and Play Store Prep

Objective: Prepare app for publishing on Google Play.
Actions:

- Generate signed release APK/AAB (using `key.properties` and Android Studio/Gradle).
- Ensure all required Play Store metadata is present (privacy policy, description).
- Verify app meets the launch checklist.
- Create release notes and versioning (update `pubspec.yaml` version).
- (Do **not** actually publish; prepare assets.)
  Expected Output: Signed app bundle and a draft Play Store entry document.
  Security: Ensure the signing key is stored securely (not in repo).
  Acceptance: Build succeeds in release mode, installation from Play Store draft works on a device.

Prompt 17 — Final Integration and Smoke Tests

Objective: Test the fully integrated system end-to-end on a “release candidate”.
Actions:

- Conduct a final end-to-end smoke test with the assembled app: sign up, verify email, join group, send chat, report, subscribe, etc.
- Test on a physical Android device for any platform issues.
- Cross-check that all modules work together (e.g. messages trigger notifications, subscription unlocks content, etc.).

- Run the emulator suite to ensure all Security Rules are passing in their final form.
  Expected Output: Checklist of all major flows tested. Bug reports for any final issues.
  Acceptance: All core flows (from SRS) work as expected with no crashes. Any bugs are minor or deferred.

Prompt 18 — Documentation and Deliverables

Objective: Produce final user/developer documentation and prepare artifacts.
Actions:

- Write a **README.md** with project overview, setup steps (including Firebase configuration), and commands to run/test.
- Document file structure and major classes/modules.
- Summarize each Claude prompt and outputs.
- Include the Security Rules file, and any SQL scripts (none in this design).
- Provide instructions for manual verification (see Section 12).
- Ensure all source code is checked in and committed in logical chunks.
  Expected Output: A `docs/` folder or similar with all documentation, plus updated README.
  Acceptance: A new developer should be able to clone repo and start dev by following these docs.

Prompt 19 — Migration and Retention Policies

Objective: Define data migration, backup, and retention procedures.
Actions:

- Draft a policy: how to handle schema changes (e.g. incremental Cloud Function on app launch to update older docs).
- Explain how to export Firestore data if needed (via `gcloud firestore export` or BigQuery).
- Set up (or schedule) periodic Firestore exports as backups on GCS.
- Specify retention: e.g. account data auto-deletion after X period.
  Expected Output: Short document on data management policy.
  Acceptance: Procedure outlined; at minimum, a sample export command is verified (not necessarily executed here).

Prompt 20 — Final Review Checklist

Objective: Verify all requirements and tests before release.
Actions:

- Run `flutter build apk` and `flutter test` one more time; record results.
- Manually verify a checklist of acceptance criteria (as below).
  Expected Output: Final checklist report (below) with [ ] for pending verification.
  Acceptance: All items in checklist are satisfied (when marked done by reviewer).

Definition-of-Done Checklist (all items initially unchecked):

- [ ] Requirements Spec complete with all FR and NFR (IDs, criteria).
- [ ] UI screens for all flows are implemented and navigable (register, login, group chat, etc.).
- [ ] Firebase backend configured (Auth, Firestore, Functions, FCM).
- [ ] User can register, verify email, login, logout. (Email flows tested.)
- [ ] User profile creation and update works; no unauthorized fields change.

- [ ] Groups can be created/joined, and membership enforced.
- [ ] Group chat working: messages sent/received in real time.
- [ ] Direct messaging implemented (two-user chat).
- [ ] Link messages require moderation before delivery.
- [ ] Moderation queue and user report flows working.
- [ ] Free content viewable by all; premium content locked without subscription.
- [ ] Subscription purchase flow implemented and verified on server side.
- [ ] Notifications delivered for incoming messages/events.
- [ ] Firestore Security Rules tested with both allowed and denied cases.
- [ ] Automated tests (unit, widget, rule tests) all passing.
- [ ] Load tests run through staging (up to target load) with performance data.
- [ ] Play Store release build signed, checks done.
- [ ] Documentation (README, developer guide) completed.
- [ ] No known critical bugs; minor issues documented.
- [ ] All citations and architecture documentation included in project docs.

Sources

- Firebase Pricing and Quotas【63†L500-L504】【54†L1-L4】【51†L1-L4】
- Firebase Architecture and Documentation (Authentication, Firestore)【60†L103-L105】
- Flutter Framework and Material Design Guidelines【61†L339-L344】
- Android Play Billing (official docs, see Android Developers site)【61†L339-L344】 (implied)
